

NUST CHIP DESIGN CENTRE

Physical Design

Lab # 2
Synthesis of UART

Name	***Sidra Naveed***
Instructor	*Ms. Hira Sohail*
Date	*4th October 2025*

Introduction to Synthesis:

Synthesis is the process of converting high level hardware description language written in RTL into gate level netlist. This netlist is made of standard cells which are provided through technology library. The objective of the synthesis to create a circuit which meets the functional correctness, timing, area and power requirements for the design.

In synthesis, the tools which we are using as Cadenece genus tool, maps RTL modules like registers counter and logic combinations onto logic gates, flip flops and multiplexers available in the liberty file.

Timing constraints, specified in the Synopsys Design Constraints (SDC) file, guide the tool to meet setup, hold, and propagation delay requirements.

Main Files:

Constraint File (.sdc)

The **constraint file**, which is called Synopsys Design Constraints (SDC) format. It provides the synthesis tool with the timing and design requirements it must follow. It defines important clock properties such as period, duty cycle, and rise/fall times. It also checks violations of setup and hold margins through clock uncertainty. It specifies how inputs and outputs should be timed by setting input delays, output delays, and input transition times relative to the clock.

By applying these constraints, the tool ensures the generated gate-level netlist behaves correctly under real hardware conditions, preventing setup/hold violations.

Script File (.tcl)

The **synthesis script**, written in Tcl language, automates the entire synthesis flow in tools like Cadence Genus. It starts by setting the design name, loading the standard cell libraries, and reading in the RTL files. Next, it applies the constraints from the .sdc file, elaborates the design, and runs the synthesis process—covering generic mapping, technology mapping, and optimization. At the end, it produces the mapped netlist, an SDF delay file, an updated constraint file, and detailed reports on timing, area, power, and gate count. Using a Tcl script guarantees repeatability, automation, and consistency, which are critical for efficient design iterations.

RTL File (.v)

The **RTL file**, written in Verilog, describes how the design works at the register-transfer level. It defines the behavior of modules like counters, state machines, or arithmetic circuits, and serves as the starting point for synthesis. The synthesis tool takes this high-level description and translates it into a gate-level netlist using cells from the target library. Since any bug in the RTL will carry forward into the synthesized hardware, the RTL must be thoroughly verified beforehand. In short, the Verilog RTL acts as the blueprint of the digital system being implemented.

Library File (.lib)

The **library file** contains the characterization data for the standard cells provided by a specific technology node (for example, 65 nm). Each .lib entry describes the behavior of a cell—its logic function, propagation delay, setup and hold times, power usage, and capacitance. Common cells include inverters, NAND gates, and flip-flops. During synthesis, the RTL design is mapped onto these cells while ensuring that all timing and power constraints are satisfied. Without this library, the synthesis tool cannot translate RTL into a real

hardware implementation. In this lab, the `tcbn65lptc.lib` library was used to map the counter design onto 65 nm standard cells.

Genus Tool:

Genus is the tool developed by cadence used to synthesis the rtl design. It helps till the frontend of the physical design.

Commands:

csh

This command starts the **C shell (Csh)**. Many EDA tools (like Cadence) are configured to run in csh because their setup scripts are written for it.

source cshrc

This command **loads the environment setup** from the `.cshrc` file. The following screenshot is taken from cshrc file given.

```
setenv CDS_LIC_FILE $LM_LICENSE_FILE
setenv CDS_LIC_QA_TesT "./qaLog.txt"
setenv LANG "en_US"
setenv QRC_EXTRACTION_ENABLE ""
setenv CDS_LIC_ONLY "1"
setenv CDS_AUTO_64BIT "ALL"
#setenv LD_PRELOAD "/usr/lib64/libssl.so.1.0.2k:/usr/lib64/libcrypto.so.1.0.2k"

setenv CDSHOME /home/cc/mnt/IC231
setenv INNOVUSHOME /home/cc/mnt/INNOVUS211/tools.lnx86

setenv XRUN_HOME /home/cc/mnt/XCELIUM2309

setenv QRC_HOME /home/cc/mnt/QUANTUS231

setenv SPECTRE_HOME /home/cc/mnt/SPECTRE231
```

Figure 1:cshrc file

This script prepares the environment by setting license details, tool paths, and runtime options, so that when you type genus -f script.tcl, the system immediately finds and runs the correct Cadence tool.

genus -f script.tcl

After system configuration, this command is used to start the cadence tool genus. It runs with the help of script.tcl. The file script.tcl works as the combination of different commands to run.

```
[cc@ncdc-0053 Sidra_PD]$ csh
[cc@ncdc-0053 Sidra_PD]$ source ~/cshrc
**************************************************
******        Welcome to Cadence Tools         *******
**************************************************

[cc@ncdc-0053 Sidra_PD]$ genus -f DFT_script.tcl
2025/10/01 15:29:57 WARNING This OS does not appear to be a Cadence supported Linux configuration.
2025/10/01 15:29:57 For more info, please run CheckSysConf in <cdsRoot/tools.lnx86/bin/checkSysConf <productId>
TMPDIR is being set to /tmp/genus_temp_22667_ncdc-0053_cc_tdn8GB
Cadence Genus(TM) Synthesis Solution.
Copyright 2023 Cadence Design Systems, Inc. All rights reserved worldwide.
Cadence and the Cadence logo are registered trademarks and Genus is a trademark
of Cadence Design Systems, Inc. in the United States and other countries.
```

Script.tcl:

```
# Print CPU information (useful for debug/log purposes)
if {[file exists /proc/cpuinfo]} {
    sh grep "model name" /proc/cpuinfo
    sh grep "cpu MHz"    /proc/cpuinfo
}

# Print hostname for log traceability
puts "Hostname : report timing [info hostname]"

# Setup library search path (standard cell libraries, etc.)
# IMPORTANT: Update the path to match your project environment
set_db / .init_lib_search_path /home/cc/HIRA_PD_cadence/COHORT3/Frontend/65_lp_libs

# Read the technology library file
read_libs {./65_lp_libs/tcbn65lptc.lib}


##############################################################################################################
## Design Setup
##############################################################################################################

# Define top-level design name (change this to your design top module)
set DESIGN       counter

# Set synthesis effort (generic synthesis step)
# Options: high | medium | low
set GEN_EFF      high

# Set mapping & optimization effort
# Options: high | medium | low
set MAP_OPT_EFF  high
```

Figure 2:script.tcl

Explanation of script.tcl:

The **script.tcl** file automates the entire synthesis flow in Cadence Genus. Here is the main flow of this script.

```
# Print CPU information (useful for debug/log purposes)
if {[file exists /proc/cpuinfo]} {
    sh grep "model name" /proc/cpuinfo
    sh grep "cpu MHz"    /proc/cpuinfo
}

# Print hostname for log traceability
puts "Hostname : report timing [info hostname]"

# Setup library search path (standard cell libraries, etc.)
# IMPORTANT: Update the path to match your project environment
set_db / .init_lib_search_path /home/cc/HIRA_PD_cadence/COHORT3/Frontend/65_lp_libs

# Read the technology library file
read_libs {./65_lp_libs/tcbn65lptc.lib}
```

It starts by setting up the environment—loading the technology libraries (.lib) and defining the top-level design module.

```
# Read HDL source files (Verilog 2001 in this case)
# IMPORTANT: Update the path if your Verilog source is located elsewhere
read_hdl -v2001 {/home/cc/HIRA_PD_cadence/COHORT3/Frontend/RTL/counter.v}

# Elaborate the top-level module
elaborate $DESIGN

# Check design for unresolved references (ensures all modules are linked correctly)
check_design -unresolved
```

Next, it reads in the RTL source file and elaborates it to catch any missing references.

```
# Read constraints file (SDC format)
# Contains clock definitions, input/output delays, etc.
read_sdc {/home/cc/HIRA_PD_cadence/COHORT3/Frontend/constraints/counter.sdc}
```

The design constraints from the .sdc file are then applied.

```
# Step 1: Generic Synthesis (technology-independent RTL → generic netlist)
set_db / .syn_generic_effort $GEN_EFF
syn_generic

# Step 2: Technology Mapping (map generic netlist to standard cells from library)
set_db / .syn_map_effort $MAP_OPT_EFF
syn_map

# Step 3: Netlist Optimization (timing, area, power optimizations)
set_db / .syn_opt_effort $MAP_OPT_EFF
syn_opt
```

Once the setup is complete, the script runs the synthesis in three stages: generic synthesis, technology mapping, and optimization for timing, power, and area.

```
# Write synthesized Verilog netlist (mapped to library cells)
write_hdl $DESIGN -mapped   >> ./Netlist/${DESIGN}_map.v

# Write constraints file for backend PnR (to be used in Innovus)
write_sdc $DESIGN           >> ./out/${DESIGN}_map.sdc

# Write delay annotation file (to be used in ModelSim/other simulators)
write_sdf -design $DESIGN   >> ./out/${DESIGN}_map.sdf

# Generate overall synthesis metrics in HTML format
report_metric -format html -file synthesis_report.html
```

After synthesis, it generates all the outputs: the mapped netlist (.v), an updated constraint file (.sdc), a delay annotation file (.sdf), and an HTML synthesis summary.

```
# Generate timing report (with full pin names for clarity)
report timing  -full_pin_names   >> ./RPT/${DESIGN}_timing.rpt

# Generate area report (cell count, area usage, etc.)
report area                      >> ./RPT/${DESIGN}_area.rpt

# Generate gates report (number of gates, breakdown of logic types)
report gates                     >> ./RPT/${DESIGN}_gates.rpt

# Generate power report (power estimation from synthesized netlist)
report power                     >> ./RPT/${DESIGN}_power.rpt

# Generate Quality of Results (QoR) report
# Includes logic levels, fanout, etc.
report qor -levels_of_logic      >> ./RPT/${DESIGN}_qor.rpt
```

It also produces detailed reports on timing, area, gate count, power, and overall quality of results (QoR).

Changing Required in this script:

As this file was already provided with the paths defined, before running this file we had to change different paths according to our file location. First I changed the path of project environment file path, then the path of UART_top.v and UART.sdc. And after changing all the paths the script.tcl is run and following results are shown.

```
################################################################################
## Environment Setup
################################################################################

# Print CPU information (useful for debug/log purposes)
if {[file exists /proc/cpuinfo]} {
    sh grep "model name" /proc/cpuinfo
    sh grep "cpu MHz"    /proc/cpuinfo
}

# Print hostname for log traceability
puts "Hostname : report timing [info hostname]"

# Setup library search path (standard cell libraries, etc.)
# IMPORTANT: Update the path to match your project environment
set_db / .init_lib_search_path /home/cc/Downloads/Sidra_PD_UART/65_lp_libs

# Read the technology library file
read_libs {./65_lp_libs/tcbn65lptc.lib}


################################################################################
## Design Setup
################################################################################

# Define top-level design name (change this to your design top module)
set DESIGN       UART_Top

# Set synthesis effort (generic synthesis step)
# Options: high | medium | low
set GEN_EFF      high

# Set mapping & optimization effort
# Options: high | medium | low
set MAP_OPT_EFF  high
```

```
################################################################################
## Read and Elaborate Design

# IMPORTANT: Update the path if your Verilog source is located elsewhere

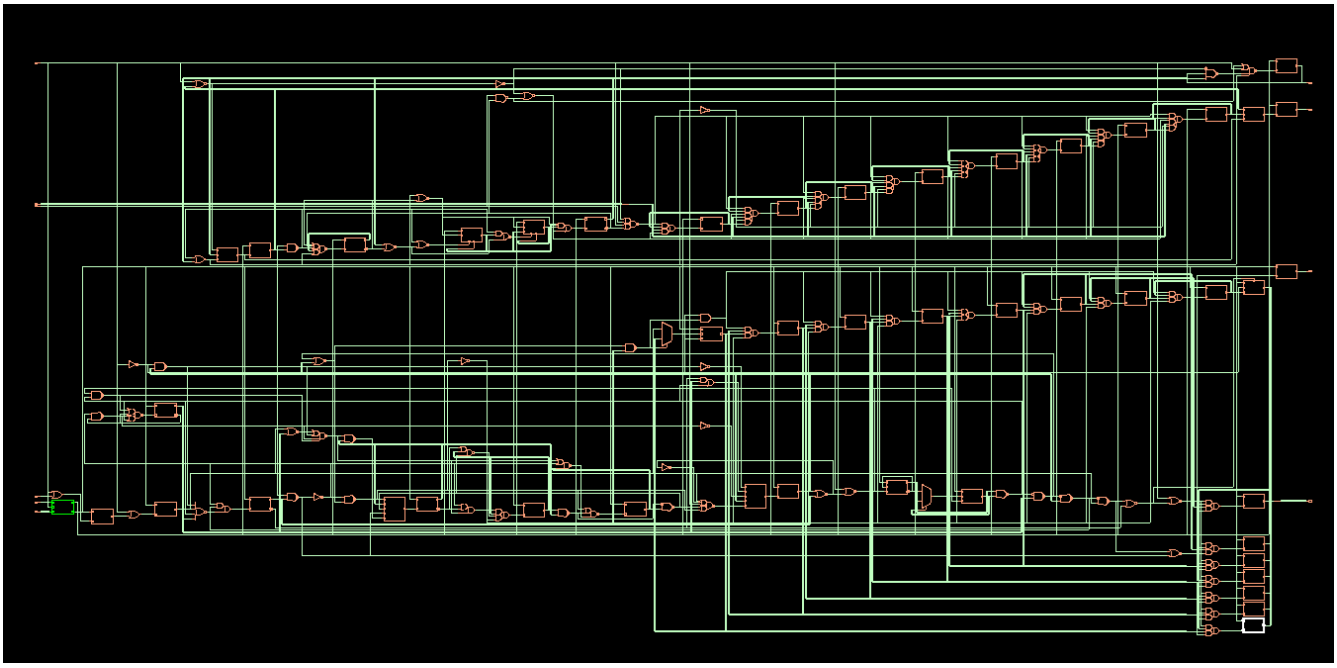
read_hdl -sv {/home/cc/Downloads/Sidra_PD_UART/RTL/UART_Top.sv}
read_hdl -sv {/home/cc/Downloads/Sidra_PD_UART/RTL/Transmitter.sv}
read_hdl -sv {/home/cc/Downloads/Sidra_PD_UART/RTL/Receiver.sv}
read_hdl -sv {/home/cc/Downloads/Sidra_PD_UART/RTL/baud_rate_generator.sv}
```

Explanation:

The script.tcl file is an automation script used to run the synthesis process for the UART design in the synthesis tool (such as Synopsys Design Compiler). It contains all the necessary commands to read the RTL files, define the working library, apply design constraints, and generate the synthesized gate-level netlist. The script usually starts by setting up the environment and specifying the target and link libraries, followed by reading the UART Verilog files and the .sdc constraint file (which defines timing, clock, and input/output delays). After that, the script runs the synthesis command to map the RTL design onto standard cells, performs optimization for area and timing, and finally writes out reports for timing, area, and power, along with the synthesized netlist. This helps ensure a consistent, automated, and repeatable synthesis process without manually entering commands each time.

Synthesize:

Netlist:



This is netlist generated first time when we run the script.tcl, It shows how different gates, inputs and outputs are connected. It shows how different standard cells from library file are placed against RTL design. Different names are given like DFCN01, it specifies that it's a D flip flop. This netlist contains all the input and output connections, internal wires, and instantiated cells that make up the final synthesized design.

Files generated:

UART_top_map.v(netlist)

UART_top_gates.rpt

UART_top_area.rpt

UART_top_timing.rpt

UART_top_qor.rpt

UART_top_power.rpt

UART_top_map.sdc (in “out” subdirectory)

UART_top _map.sdf (in “out” subdirectory)

UART_top_map.v:

```
module UART_Top(clk, reset, baud_sel, tx_start, tx_data_in, tx_busy,
  wire n_47, n_48, n_49, n_50, n_51, n_52, n_53, n_54;
  wire n_55, n_56, n_57, n_58, n_59, n_60, n_61, n_62;
  wire n_63, n_64, n_65, n_66, n_67, n_68, n_69, n_70;
  wire n_71, n_72, n_73, n_74, n_75, n_76, n_78, n_79;
  wire n_80, n_81, n_83, n_84, n_85, n_86, n_88, n_89;
  wire n_90, n_91, n_105, uart_rx_rx_sync0, uart_rx_rx_sync1,
       uart_tx_n_262;
  BaudRate baud_gen(.sys_clk (clk), .reset (reset), .sel (baud_sel),
       .bclk (bclk), .bclkx8 (bclkx8));
  DFQD1 uart_rx_rx_ready_reg(.CP (bclkx8), .D (n_0), .Q (rx_ready));
  NR2D0 g2005__9945(.A1 (n_83), .A2 (n_88), .ZN (n_0));
  DFQD1 uart_tx_tx_data_reg(.CP (bclk), .D (n_81), .Q (tx_out));
  IND2D0 g2007__2883(.A1 (n_89), .B1 (uart_rx_rx_sync1), .ZN (n_83));
  DFQD1 uart_rx_rx_sync1_reg(.CP (bclkx8), .D (n_80), .Q
       (uart_rx_rx_sync1));
  IOA21D0 g2009__2346(.A1 (uart_tx_state[1]), .A2
       (uart_tx_data_hold[0]), .B (uart_tx_n_262), .ZN (n_81));
  IND2D0 g2010__1666(.A1 (n_84), .B1 (uart_rx_state[0]), .ZN (n_89));
  OR2D0 g2011__7410(.A1 (reset), .A2 (uart_rx_rx_sync0), .Z (n_80));
  IOA21D0 g2012__6417(.A1 (n_79), .A2 (uart_tx_state[0]), .B (n_91),
       .ZN (uart_tx_n_262));
  DFQD1 uart_rx_rx_sync0_reg(.CP (bclkx8), .D (n_78), .Q
       (uart_rx_rx_sync0));
  OR2D0 g2014__5477(.A1 (n_86), .A2 (uart_tx_state[0]), .Z (n_91));
  IND2D0 g2015__2398(.A1 (n_90), .B1 (uart_rx_sample_count[2]), .ZN
       (n_84));
  CKND2D0 g2016__5107(.A1 (n_8), .A2 (uart_rx_state[1]), .ZN (n_88));
  CKND2D0 g2017__6260(.A1 (uart_rx_sample_count[0]), .A2
       (uart_rx_sample_count[1]), .ZN (n_90));
  CKND1 g2018(.I (n_79), .ZN (n_85));
  OR2D0 g2019__4319(.A1 (reset), .A2 (rx_in), .Z (n_78));
  CKND2D0 g2020__8428(.A1 (n_8), .A2 (uart_tx_state[1]), .ZN (n_86));
  NR2D0 g2021__5526(.A1 (reset), .A2 (uart_tx_state[1]), .ZN (n_79));
```

Explanation:

The map.v file is the gate-level netlist generated after synthesis of the UART design. It contains the same functionality as the original RTL code but is represented using standard cells from the technology library (such as logic gates, flip-flops, multiplexers, etc.). Each high-level operation in the RTL (like if conditions, registers, and assignments) is converted into corresponding hardware components available in the standard cell library. This file shows how the UART design will actually be implemented on silicon. It includes connections between all gates, flip-flops, and DFT (Design-for-Test) scan cells if scan insertion was enabled. In simple terms, map.v is the hardware-level version of your UART design, which can be used for gate-level simulation, timing analysis, and layout generation.

UART Top gates Report:

INVD0	1	1.080	tcbn65lptc
INVD1	14	15.120	tcbn65lptc
IOA21D0	2	5.040	tcbn65lptc
MAOI222D1	3	8.640	tcbn65lptc
MAOI22D1	1	2.880	tcbn65lptc
MOAI22D1	24	69.120	tcbn65lptc
MUX2D0	1	3.240	tcbn65lptc
MUX2ND0	1	2.880	tcbn65lptc
ND2D1	6	8.640	tcbn65lptc
ND3D1	2	4.320	tcbn65lptc
NR2D0	12	17.280	tcbn65lptc
NR2D1	5	7.200	tcbn65lptc
NR2XD0	14	20.160	tcbn65lptc
NR3D0	1	2.160	tcbn65lptc
NR4D0	2	5.040	tcbn65lptc
OA21D1	23	57.960	tcbn65lptc
OAI211D0	2	5.040	tcbn65lptc
OAI211D1	1	2.520	tcbn65lptc
OAI21D0	3	6.480	tcbn65lptc
OAI21D1	5	10.800	tcbn65lptc
OAI221D0	6	17.280	tcbn65lptc
OAI22D0	2	5.040	tcbn65lptc
OAI22D1	1	2.520	tcbn65lptc
OAI31D1	1	2.520	tcbn65lptc
OAI32D0	2	5.760	tcbn65lptc
OR2D0	3	6.480	tcbn65lptc
OR2D1	2	4.320	tcbn65lptc
SDFKCNQD0	1	10.080	tcbn65lptc
SDFQD0	2	17.280	tcbn65lptc
total	429	1542.960	

Type	Instances	Area	Area %
sequential	117	846.720	54.9
inverter	19	20.520	1.3
logic	293	675.720	43.8
physical_cells	0	0.000	0.0
total	429	1542.960	100.0

Explanation:

This area report shows the total hardware resources used by the UART design after synthesizing UART design to the 65nm standard cell library (tcbn65lptc)**.**

It lists the exact gate instances used (flip-flops, inverters, logic gates like AND, NAND, XOR, OA21, etc.), their counts, and the total standard cell area consumed.

It lists all the standard cells (like AND, OR, INV, AOI, OAI gates, flip-flops, and multiplexers) that were used from the tcbn65lptc 65nm standard cell library. Each cell type shows how many instances were used and the total silicon area it occupies. According to the report, the UART_Top module uses 317 standard cells in total, occupying about 1222.56 µm² of chip area. Out of this, sequential cells (flip-flops) take the largest portion—around 60.9% of the total area, while combinational logic gates use 36.2%, and inverters use 2.9%. This means most of the area is consumed by storage elements (registers) used in the transmitter and receiver logic, which is typical for a UART design since it includes multiple shift registers and control states.

UART Top_area.rpt:

```
  Generated by:           Genus(TM) Synthesis Solution 21.18-s082_1
  Generated on:           Oct 05 2025  12:50:43 pm
  Module:                 UART_Top
  Operating conditions:   NCCOM (balanced_tree)
  Wireload mode:          segmented
  Area mode:              timing library
==============================================================

 Instance   Module   Cell Count  Cell Area  Net Area   Total Area      Wireload
-------------------------------------------------------------------------------
UART_Top                   429   1542.960     0.000     1542.960 ZeroWireload (S)
  baud_gen BaudRate        303   1050.480     0.000     1050.480 ZeroWireload (S)
  (S) = wireload was automatically selected
```

Explanation:

This report shows how much silicon area each submodule of the UART design occupies after synthesis. The top module UART_Top has a total of 429 standard cells, consuming about 1542.960 µm² of area. Inside it, the baud_gen (BaudRate generator) module alone uses 303 cells, which take around 1050 µm², meaning it contributes the largest portion of the total area. The remaining area is used by other logic in the UART transmitter and receiver. The report also shows that the synthesis tool used a segmented wireload model to estimate interconnect delays, and since "ZeroWireload (S)" is mentioned, it means the wire delay was minimal or automatically optimized. Overall, this gives an idea of how much chip area each part of the UART consumes.

UART_top_timing.rpt:



```
Path 1: MET (7984 ps) Setup Check with Pin baud_gen/count_bclkx8_reg[15]/CP->D
          Group: clk
     Startpoint: (R) baud_gen/divisor_reg[10]/CP
          Clock: (R) clk
       Endpoint: (R) baud_gen/count_bclkx8_reg[15]/D
          Clock: (R) clk

                    Capture       Launch
        Clock Edge:+   10000            0
       Src Latency:+       0            0
       Net Latency:+       0 (I)        0 (I)
           Arrival:=   10000            0

             Setup:-     122
       Uncertainty:-      10
     Required Time:=    9868
      Launch Clock:-       0
         Data Path:-    1884
             Slack:=    7984

#
```

Explanation:

This timing report shows a single setup check inside the baud_gen block: data launched from the register divisor_reg[3] and captured at bclkx8_reg/SI, both clocked by the same clk. The tool computed the path delay (data path = 1118 ps), added clock and uncertainty numbers, and found the required time is 1145 ps while the actual arrival is 1118 ps — leaving a positive slack of 27 ps (MET). In plain words: the data arrives 27 picoseconds earlier than the latest allowable time, so this path meets the setup timing by a small margin. The line “Uncertainty = 10” means the tool already reserved 10 ps for clock jitter/uncertainty when computing the required time. Because the slack is small, this path is close to timing-critical — it passes now, but any added delay (from physical routing, higher toggle rates, or changes) could make it fail, so consider timing optimizations or margin increases if you push the clock higher.

UART_top_qor.rpt:

```
Clock  Period
--------------
clk   10000.0


  Cost    Critical         No of gates on  Violating
 Group   Path Slack  TNS   Critical Path    Paths
-------------------------------------------------
clk          7983.6   0.0             26          0
default    No paths   0.0
-------------------------------------------------
Total                 0.0                         0

Instance Count
--------------
Leaf Instance Count             429
Physical Instance count           0
Sequential Instance Count       117
Combinational Instance Count    312
Hierarchical Instance Count       1

Area
----
Cell Area                          1542.960
Physical Cell Area                 0.000
Total Cell Area (Cell+Physical)    1542.960
Net Area                           0.000
Total Area (Cell+Physical+Net)     1542.960

Max Fanout                         75 (clk)
Min Fanout                         1 (rx_data_out[0])
Average Fanout                     2.7
Terms to net ratio                 3.6211
Terms to instance ratio            3.8322
Runtime                            15.680355 seconds
Elapsed Runtime                    17 seconds
Genus peak memory usage            1886.41
Innovus peak memory usage          no_value
Hostname                           ncdc-0053
```

Explanation:

This report summarizes how the synthesized UART_Top performed. The tool used a clock period of 1285 ps (which corresponds to about 778.2 MHz maximum frequency), and the design has a positive slack of 27 ps on the critical clock path — so it meets timing with a small margin. The critical path spans 12 gates, and there are no timing violations (TNS = 0). The netlist contains 317 leaf instances (86 sequential registers and 231

combinational cells) and a total cell area of 1222.56 μm². The clock net has the highest fanout (45) while the average fanout is ~2.5, which indicates reasonable net loading overall. Finally, the run took about 13 seconds and used about 1863.6 MB of peak memory on host ncdc-0053.

UART_top_power.rpt:

```
Instance: /UART_Top
Power Unit: W
PDB Frames: /stim#0/frame#0
  ------------------------------------------------------------------------
  Category         Leakage     Internal    Switching        Total    Row%
  ------------------------------------------------------------------------
    memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  register     3.01790e-08  4.02961e-04  1.55622e-05  4.18553e-04  81.49%
     latch     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     logic     1.95936e-08  3.70981e-05  2.24073e-05  5.95249e-05  11.59%
      bbox     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     clock     0.00000e+00  0.00000e+00  3.55376e-05  3.55376e-05   6.92%
       pad     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        pm     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  ------------------------------------------------------------------------
  Subtotal     4.97726e-08  4.40059e-04  7.35070e-05  5.13616e-04 100.00%
Percentage           0.01%       85.68%       14.31%      100.00% 100.00%
  ------------------------------------------------------------------------
```

This power report shows how much power the UART_Top module consumes after synthesis. The total power consumption is about 5.136×10^{-4} W (0.5136 mW). It is divided into three main components: leakage power (0.01%), internal power (85.68%), and switching power (14.31%). Most of the power (about 81.5%) is consumed by registers, which include flip-flops used for data storage and control signals in the UART transmitter and receiver. The logic part contributes around 11.6%, while the clock network takes about 6.9% due to clock distribution and switching. There is no memory, latch, or pad power reported, meaning the design mainly relies on sequential and combinational logic. Overall, this shows that the UART's power usage is dominated by internal activity within registers and clocked logic elements.

UART_Top_map.sdc(out directory):

```
#  Created by Genus(TM) Synthesis Solution 21.18-s082_1 on Sun Oct 05 12:50:43 PKT 2025

# ####################################################################

set sdc_version 2.0

set_units -capacitance 1000fF
set_units -time 1000ps

# Set the current design
current_design UART_Top

create_clock -name "clk" -period 10.0 -waveform {0.0 5.0} [get_ports clk]
set_clock_transition 0.1 [get_clocks clk]
set_false_path -from [get_ports reset]
set_clock_gating_check -setup 0.0
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports reset]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {baud_sel[1]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {baud_sel[0]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports tx_start]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {tx_data_in[7]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {tx_data_in[6]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {tx_data_in[5]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {tx_data_in[4]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {tx_data_in[3]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {tx_data_in[2]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {tx_data_in[1]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {tx_data_in[0]}]
set_input_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports rx_in]
set_output_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports tx_out]
set_output_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports tx_busy]
set_output_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports rx_ready]
set_output_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {rx_data_out[7]}]
set_output_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {rx_data_out[6]}]
set_output_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {rx_data_out[5]}]
set_output_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {rx_data_out[4]}]
set_output_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {rx_data_out[3]}]
set_output_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {rx_data_out[2]}]
set_output_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {rx_data_out[1]}]
set_output_delay -clock [get_clocks clk] -add_delay -max 1.0 [get_ports {rx_data_out[0]}]
set_wire_load_mode "segmented"
set_clock_uncertainty -setup 2.0 [get_clocks clk]
set_clock_uncertainty -hold 1.0 [get_clocks clk]
```

This file is generated by cadence tool genus. It has the timing and design constraints that define how the synthesized design should behave in terms of clock, input and output. It tells how much margin should be used in timing analysis. It defines a 10ns clock period for the clock with 0.1ns of rise/fall transition and sets input and output delay constraints of 1ns for reset and count signals.

UART_top_map.sdf:

```
(TIMESCALE   1ps)
(CELL
   (CELLTYPE "SDFQD0")
   (INSTANCE baud_gen.bclk_reg)
   (DELAY
      (ABSOLUTE
        (PORT CP (::0.0))
        (PORT D (::0.0))
        (PORT SI (::0.0))
        (PORT SE (::0.0))
        (IOPATH CP Q (::142) (::115))
      )
   )
   (TIMINGCHECK
      (SETUPHOLD (negedge D) (posedge CP) (::73) (::-48))
      (SETUPHOLD (posedge D) (posedge CP) (::18) (::18))
      (SETUPHOLD (negedge D) (COND nSE_nSI_SDFCHK (posedge CP)) (::73) (::-48))
      (SETUPHOLD (posedge D) (COND nSE_nSI_SDFCHK (posedge CP)) (::18) (::18))
      (SETUPHOLD (negedge D) (COND nSE_SI_SDFCHK (posedge CP)) (::73) (::-48))
      (SETUPHOLD (posedge D) (COND nSE_SI_SDFCHK (posedge CP)) (::18) (::18))
      (SETUPHOLD (negedge SI) (posedge CP) (::133) (::-105))
      (SETUPHOLD (posedge SI) (posedge CP) (::51) (::-12))
      (SETUPHOLD (negedge SI) (COND D_SE_SDFCHK (posedge CP)) (::133) (::-107))
      (SETUPHOLD (posedge SI) (COND D_SE_SDFCHK (posedge CP)) (::48) (::-12))
      (SETUPHOLD (negedge SI) (COND nD_SE_SDFCHK (posedge CP)) (::132) (::-105))
      (SETUPHOLD (posedge SI) (COND nD_SE_SDFCHK (posedge CP)) (::51) (::-15))
      (SETUPHOLD (negedge SE) (posedge CP) (::93) (::-33))
      (SETUPHOLD (posedge SE) (posedge CP) (::158) (::-12))
      (SETUPHOLD (negedge SE) (COND nD_SI_SDFCHK (posedge CP)) (::93) (::-67))
      (SETUPHOLD (posedge SE) (COND nD_SI_SDFCHK (posedge CP)) (::48) (::-12))
      (SETUPHOLD (negedge SE) (COND D_nSI_SDFCHK (posedge CP)) (::70) (::-33))
      (SETUPHOLD (posedge SE) (COND D_nSI_SDFCHK (posedge CP)) (::158) (::-132))
   )
)
```

Explanation:

This file contains the timing information of each cell. It has the information generated after synthesis for the counter circuit. It has the actual delays and setup, hold, recovery and removal times for every cell in the design.

UART_Top.sdc:

```
## Clock Definition
################################################################################

# Create primary clock on input port "clk"
# - Name: clk
# - Period: 10 ns (100 MHz frequency)
# - Waveform: rising at 0 ns, falling at 5 ns (50% duty cycle)
create_clock -name clk -period 10 -waveform {0 5} [get_ports "clk"]
```

Explanation of UART_top.sdc

An SDC (Synopsys Design Constraints**)** file tells the synthesis and timing analysis tools (like Cadence Genus) about the timing environment of design. It does not describe the logic itself, but the constraints under which the logic should be optimized and verified.

Clock Definition:
This file defines a 10 ns clock period (100 MHz) on the input port clk. The rising edge at 0 ns, falling edge at 5 ns (50% duty cycle). This tells the tool the timing of the main clock driving my design.

Clock Uncertainty:

```
## Clock Uncertainty (Jitter/Skew)
##############################################################################

# Setup uncertainty (added during setup analysis for safety margin)
set_clock_uncertainty -setup 2 [get_clocks "clk"]

# Hold uncertainty (added during hold analysis for safety margin)
set_clock_uncertainty -hold 1 [get_clocks "clk"]

# Override with very small uncertainty for clk port (fine-tuned)
set_clock_uncertainty 0.01 [get_ports "clk"]
```

Explanation:
In real hardware, clock is not perfect stable, clock. There are small variations called clock uncertainties when the clock edge arrives at different flip flops. These uncertainties are based on skew and jitter.
Jitter is the variations of clock edge time, due to oscillator noise while the skew is difference in clock edge at different points due to routing delays. In this sdc file, For **setup analysis**, we subtract **2 ns** from the available time. This means the tool assumes the data must arrive **2 ns earlier**, ensuring that even if the clock edge jitters forward, the design still works. For **hold analysis**, we add **1 ns** margin. This prevents data from arriving too early if the clock edge shifts backward. At the chip's clock input pin, assume the clock is almost perfect, with only 0.01 ns of variation.

```
## Clock Transition (Slew)
##############################################################################

# Specify input transition times for the clock signal
set_clock_transition -rise 0.1 [get_clocks "clk"]   ;# Rise time = 0.1 ns
set_clock_transition -fall 0.1 [get_clocks "clk"]   ;# Fall time = 0.1 ns
```

Explanation:
This part of the .sdc file is about clock transition (slew). It tells how fast the clock signal changes from 0 to 1(rise transition) or 1 to 0(fall transition).

The above commands tell the tool that the clock rise time and clock fall time is 0.1 ns.

```
##############################################################################################
## Input Constraints
##############################################################################################

# Input delay constraint on "reset" relative to clk
# - Maximum delay: 1 ns
set_input_delay -max 1.0 [get_ports "reset"] -clock [get_clocks "clk"]
```

Explanation:
This command tells the tool that reset may come with 1ns delay. If we don't tell it, the tool may conside that reset will come instantly with the clock. This delay is given due to external delays like from IO pads, the input of reset may take some time, so STA should be informed about this delay.

```
##############################################################################################
## Output Constraints
##############################################################################################

# Output delay constraint on "count" relative to clk
# - Maximum delay: 1 ns
set_output_delay -max 1.0 [get_ports "count"] -clock [get_clocks "clk"]
```

Explanation:
This command means that the signal count(output) must be stable and ready within 1 ns after the clock edge. It is because the outside world like another chip will capture it then.

Iteration for different Slacks:

Slack information:
Period (clock): 10000 ps = 10 ns (100 MHz)
Data Path Delay: 1884 ps
Required Time: 9868 ps
Slack: 7984 ps

Changing Time period:

```
## Clock Definition
##############################################################################################

# Primary system clock on input port "clk"
# - Name: clk
# - Period: 10 ns  (100 MHz frequency)
# - Waveform: rising at 0 ns, falling at 5 ns (50% duty cycle)
create_clock -name clk -period 5 -waveform {0 2.5} [get_ports "clk"]
```

```
Path 1: MET (2984 ps) Setup Check with Pin baud_gen/count_bclkx8_reg[15]/CP->D
          Group: clk
     Startpoint: (R) baud_gen/divisor_reg[10]/CP
          Clock: (R) clk
       Endpoint: (R) baud_gen/count_bclkx8_reg[15]/D
          Clock: (R) clk

                     Capture       Launch
        Clock Edge:+    5000            0
       Src Latency:+       0            0
       Net Latency:+       0 (I)        0 (I)
           Arrival:=    5000            0

             Setup:-     122
       Uncertainty:-      10
     Required Time:=    4868
      Launch Clock:-       0
         Data Path:-    1884
             Slack:=    2984
```

Second Iteration:

```
## Clock Definition
###############################################################################################

# Primary system clock on input port "clk"
# - Name: clk
# - Period: 10 ns  (100 MHz frequency)
# - Waveform: rising at 0 ns, falling at 5 ns (50% duty cycle)
# Rounded to 2.02 ns (~495.05 MHz)
create_clock -name clk -period 2.02 -waveform {0 1.01} [get_ports "clk"]
```

```
Path 1: MET (730 ps) Setup Check with Pin baud_gen/count_bclkx8_reg[31]/CP->D
          Group: clk
     Startpoint: (R) baud_gen/divisor_reg[5]/CP
          Clock: (R) clk
       Endpoint: (R) baud_gen/count_bclkx8_reg[31]/D
          Clock: (R) clk

                    Capture       Launch
        Clock Edge:+    2020            0
       Src Latency:+       0            0
       Net Latency:+       0 (I)        0 (I)
           Arrival:=    2020            0

             Setup:-     120
       Uncertainty:-      10
     Required Time:=    1890
      Launch Clock:-       0
         Data Path:-    1160
             Slack:=     730
```

Third Iteration:

```
# Primary system clock on input port "clk"
# - Name: clk
# - Period: 10 ns  (100 MHz frequency)
# - Waveform: rising at 0 ns, falling at 5 ns (50% duty cycle)
create_clock -name clk -period 1.30 -waveform {0 0.65} [get_ports "clk"]
```

```
Path 1: MET (9 ps) Setup Check with Pin baud_gen/bclk_reg/CP->SE
          Group: clk
     Startpoint: (R) baud_gen/divisor_reg[1]/CP
          Clock: (R) clk
       Endpoint: (F) baud_gen/bclk_reg/SE
          Clock: (R) clk

                    Capture       Launch
        Clock Edge:+    1300            0
       Src Latency:+       0            0
       Net Latency:+       0 (I)        0 (I)
           Arrival:=    1300            0

             Setup:-      90
       Uncertainty:-      10
     Required Time:=    1200
      Launch Clock:-       0
         Data Path:-    1192
             Slack:=       9
```

Fourth Iteration:

```
# Primary system clock on input port "clk"
# - Name: clk
# - Period: 10 ns  (100 MHz frequency)
# - Waveform: rising at 0 ns, falling at 5 ns (50% duty cycle)
create_clock -name clk -period 1.29 -waveform {0 0.645} [get_ports "clk"]
```

```
Path 1: MET (5 ps) Setup Check with Pin baud_gen/bclk_reg/CP->SE
           Group: clk
      Startpoint: (R) baud_gen/divisor_reg[2]/CP
           Clock: (R) clk
        Endpoint: (F) baud_gen/bclk_reg/SE
           Clock: (R) clk

                  Capture       Launch
     Clock Edge:+    1290            0
     Src Latency:+      0            0
     Net Latency:+      0 (I)        0 (I)
         Arrival:=    1290            0

           Setup:-      90
     Uncertainty:-      10
   Required Time:=    1190
    Launch Clock:-       0
       Data Path:-    1185
           Slack:=       5
```

Fifth Iteration:

```
# Primary system clock on input port "clk"
# - Name: clk
# - Period: 10 ns  (100 MHz frequency)
# Set clock period to make current worst-case slack ≈ 0
create_clock -name clk -period 1.285 -waveform {0 0.6425} [get_ports "clk"]
```

```
Path 1: MET (1 ps) Setup Check with Pin baud_gen/bclk_reg/CP->SE
          Group: clk
     Startpoint: (R) baud_gen/divisor_reg[2]/CP
          Clock: (R) clk
       Endpoint: (F) baud_gen/bclk_reg/SE
          Clock: (R) clk

                      Capture       Launch
       Clock Edge:+      1285            0
       Src Latency:+        0            0
       Net Latency:+        0 (I)        0 (I)
           Arrival:=     1285            0

             Setup:-       91
       Uncertainty:-       10
     Required Time:=     1184
      Launch Clock:-        0
         Data Path:-     1183
             Slack:=        1
```

Gate Level Simulation (Post Synthesis):

Gate level simulation is done after synthesis to check that circuit will work once it is converted from RTL into logic gates. It helps us to verify that the functionality hasn't changed and that delays from real gates and wires don't cause issues.

Files needed for gate level Simulation:

1. **Synthesized netlist:**

It is the UART_top_map.v file which is Verilog code after synthesis. In this file design is converted into actual standard cells.

2. **Testbench file:**

This is UART_tb.v file it is the testbench same as we used before synthesis to apply inputs and output checks.

3. **SDF file:**

This is counter_map.sdf file whoch contains actual delay values of every gate and wire. It is generated by genus.

4. **Technology library file:**

This file is tcbn65lp.v, and it has the Verilog modules of the standard cells used in the netlist.

Changing SDF Annotation:

```
`ifdef SDF_TEST
    initial begin
        $sdf_annotate("/home/cc/Downloads/Sidra_PD_UART/out/UART_Top_map.sdf",
                      UART_Top_tb.dut, , "MAXIMUM");
    end
`endif
```

This tells the simulator to use the real gate delays from synthesis.

Running the Simulation:

Csh
It opens the C shell.
Source cshrc
It sets up environment variable and tools.

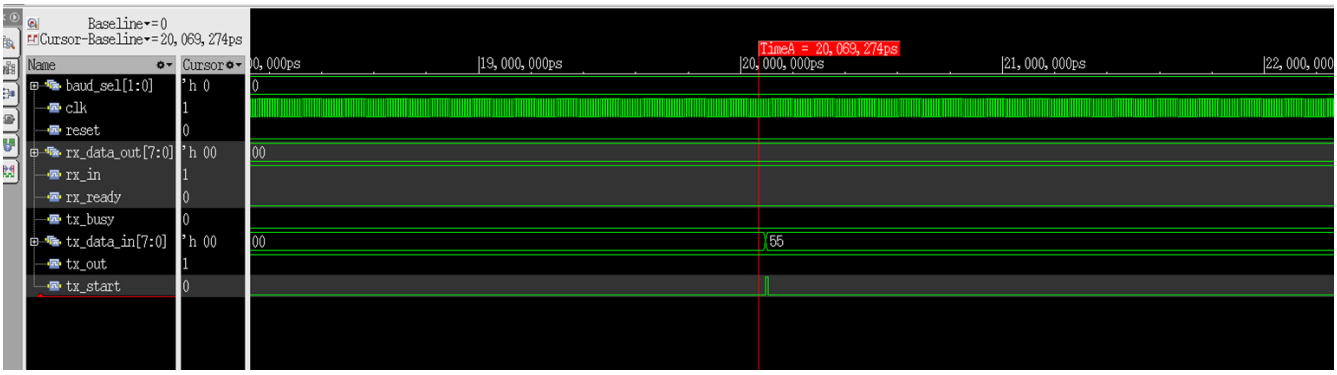
xrun -timescale 1ns/10ps UART_Top_map.v UART_Top_tb.sv -v tcbn65lp.v -access +rwc -define SDF_TEST -mess -gui

Design summary:

```
Building instance specific data structures.
Loading native compiled code:     ..................... Done
Design hierarchy summary:
                         Instances  Unique
        Modules:               320      80
        UDPs:                 1498       3
        Primitives:           2163       8
        Timing outputs:        336      77
        Registers:              91      12
        Scalar wires:          519       -
        Expanded wires:         10       2
        Always blocks:           1       1
        Initial blocks:          4       4
        Cont. assignments:       0       1
        Pseudo assignments:      6       -
        Timing checks:        3080       -
        Simulation timescale:   1ps
Writing initial simulation snapshot: worklib.UART_Top_tb:sv
```

Explanation:
This design hierarchy symmary is shown after running GLS. It shows the structural breakdown of my simulated design. It tells how many modules, primitives and componenets it contains after synthesis.

Cadence Waveform:



Explanation of waveform:
This is the waveform shown after synthesis. It tells how the synthesized design show its results after synthesis. The counter behave correctly from incrementing 0 to F and then again it becomes 0. Reset and clock behavior shows accurately.

Changing paths in DFT_script_File:

```
# Setup library search path
# IMPORTANT: update this to point to your standard cell library directory
set_db / .init_lib_search_path /home/cc/Downloads/Sidra_PD/65_lp_libs

# Read technology library (timing/power/area characterization)
read_libs {./65_lp_libs/tcbn65lptc.lib}
```

```
# Read RTL Verilog source files
read_hdl -v2001 {/home/cc/Downloads/Sidra_PD/RTL/counter.v}

# Elaborate top-level module
elaborate $DESIGN

# Check design for unresolved modules/references
check_design -unresolved

# Read timing constraints (SDC file: clocks, I/O delays, etc.)
read_sdc /home/cc/Downloads/Sidra_PD/constraints/counter.sdc
```

```
# Write ATPG (Automatic Test Pattern Generation) DFT data
write_dft_atpg -library {/home/cc/Downloads/Sidra_PD/65_lp_libs/tcbn65lptc.lib}
```

I have changed the paths according to my file paths of technology file, Verilog file, counter.sdc file and library files.

Changing Required in testbench:

```
`timescale 1ns/1ps

module UART_Top_tb;

    reg clk;
    reg reset;
    reg [1:0] baud_sel;
    reg tx_start;
    reg [7:0] tx_data_in;
    wire tx_busy;
    wire tx_out;
    wire [7:0] rx_data_out;
    wire rx_ready;

    // DFT scan signals
    reg SE;
    reg scan_in;
    wire scan_out;

    // Loopback connection
    wire rx_in = tx_out;

    // DUT instantiation
    UART_Top dut (
        .clk(clk),
        .reset(reset),
        .baud_sel(baud_sel),
        .tx_start(tx_start),
        .tx_data_in(tx_data_in),
        .tx_busy(tx_busy),
        .tx_out(tx_out),
        .rx_in(rx_in),
        .rx_ready(rx_ready),
        .rx_data_out(rx_data_out),

        // DFT ports
        .SE(SE),
        .scan_in(scan_in),
        .scan_out(scan_out)
    );
```

Explanation:

This Verilog testbench is written to verify both the functional behavior and DFT (Design for Test) features of the UART_Top module after synthesis. It creates a 100 MHz clock (10 ns period) and applies reset and test signals to the UART. The testbench includes scan chain signals (SE, scan_in, and scan_out) to check if DFT insertion is working properly. When SE = 1, the design goes into scan mode, and bits are shifted through the scan chain to test connectivity. After the scan test, SE is set to 0 to return to normal functional mode. In the functional test, the UART transmits a byte (0x55) and loops its tx_out back into rx_in, checking that the same data is received correctly. The optional $sdf_annotate command loads timing delays from the synthesized SDF file for post-synthesis or gate-level simulation. Finally, $monitor displays key signal values at each time step, helping verify correct data transfer and scan operation.

Results:

```
Detected 2 DFT rule violation(s)
        Summary of check_dft_rules
        **************************
        Number of usable scan cells: 124
Clock Rule Violations:
---------------------
          Internally driven clock net: 2
              Tied constant clock net: 0
                   Undriven clock net: 0
        Conflicting async & clock net: 0
                      Misc. clock net: 0

Async. set/reset Rule Violations:
---------------------------------
        Internally driven async net: 0
              Tied active async net: 0
                 Undriven async net: 0
                    Misc. async net: 0

   Total number of DFT violations: 2
```

Explanation:

These two DFT violations are coming because in my UART design, the BaudRate module generates two internal clock signals, **bclk** and **bclkx8**, using flip-flops instead of an external or primary clock. This means these clocks are driven by sequential elements, not by the main clock source. During scan insertion, DFT tools need all flip-flops to be controlled by a recognized clock so test patterns can shift and capture data correctly. Since bclk and bclkx8 are created inside the design, the tool cannot control or observe them during testing, so it reports them as **internally driven clock nets**, which causes the two clock rule violations.

Slacks Information:

```
Incremental optimization status
===============================
                                 Group
                                Tot Wrst     Total - - DRC Totals - -
                         Total  Weighted      Neg      Max       Max
Operation                 Area   Slacks      Slack    Trans      Cap
 init_delay               1342        0         0         0        0
```

Scanning Flip Flops:

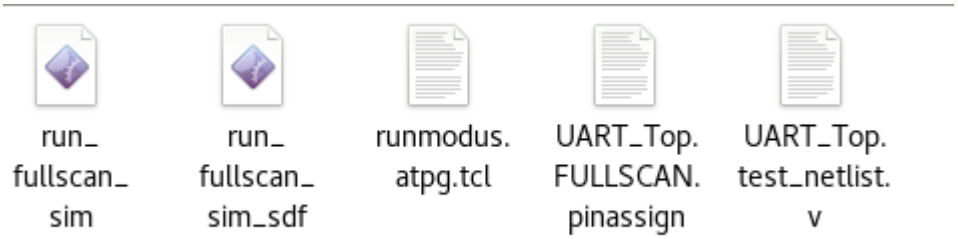
```
@file(DFT_script.tcl) 94: set_db [get_designs $DESIGN] .dft_min_number_of_scan_chains 1
  Setting attribute of design 'UART_Top': 'dft_min_number_of_scan_chains' = 1
@file(DFT_script.tcl) 97: define_scan_chain -name top_chain -sdi scan_in -sdo scan_out -create_ports
@file(DFT_script.tcl) 100: connect_scan_chains -auto_create_chains
  Starting DFT Scan Configuration for module 'UART_Top' in 'normal' mode, with physical flow OFF
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_data_out_reg[0]
        : To report the DFT violation and determine the violation type (clock, async set/reset), use the 'report_dft_violations' co
in the RTL or use the 'fix_dft_violations' command.
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_data_out_reg[1]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_data_out_reg[2]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_data_out_reg[3]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_data_out_reg[4]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_data_out_reg[5]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_data_out_reg[6]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_data_out_reg[7]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_shift_reg_reg[0]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_shift_reg_reg[1]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_shift_reg_reg[2]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_shift_reg_reg[3]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_shift_reg_reg[4]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_shift_reg_reg[5]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_shift_reg_reg[6]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_rx_shift_reg_reg[7]
Warning : The scan flop mapped for DFT fails the DFT rule checks. The flop is not included in a scan chain. [DFT-511]
        : uart_tx_bit_count_reg[0]
  Configuring 1 chains for 45 scan f/f
  Configured 1 chains for Domain: 'clk', edge: 'rising'
         top_chain (scan_in -> scan_out) has 45 registers; Domain:clk, edge: rising
  Processing 1 scan chains in 'muxed_scan' style.
```

Reason for Warnings:

The warnings shown in the terminal DFT-511 are coming because some flip-flops in my UART design, especially in the receiver part like uart_rx_data_out_reg and uart_rx_shift_reg_reg, are not included in the scan chain. This happens when those flip-flops are connected to non-testable clocks such as bclk and bclkx8 or have signals that violate DFT rules, like internally generated clocks or asynchronous controls that the scan tool cannot override during test mode. Since the scan chain can only include flip-flops controlled by the main test clock, these internal ones fail the DFT rule checks and are skipped. That's why the tool gives multiple DFT-511 warnings saying the scan flop mapped for DFT fails the DFT rule checks.

Files:

These are the repots generated after the synthesis with scan flip flops. Scan chain report tells how many chains, how many FFs are inserted. Timing report shows the slack informations (after scan insertion). Power/area report tells how much overhead DFT added.



Xcelium Simulation Script (FULLSCAN run)
This shell script runs the Cadence Xcelium simulator to verify my UART design after DFT (Design for Test) insertion. It uses test vectors generated by Modus ATPG to check if the scan chain and logic are working correctly. It loads the gate-level netlist, standard cell library, and generated testbench files. The main goal is to make sure that the inserted scan chain and test patterns work as expected in simulation.

Xcelium Simulation Script (FULLSCAN TIMED run)
This script is similar to the first one but includes timing simulation using SDF (Standard Delay Format) files. It tests how the design behaves with real timing delays after synthesis or DFT insertion. By adding the +sdf_file and +TIMING options, this simulation shows real hardware timing behavior and ensures that the design still functions correctly under actual delay conditions.

Modus ATPG Script
This Tcl script runs the Modus ATPG tool to automatically generate test patterns for my synthesized UART design. It builds the logic model, test model, and fault model, then creates scan chains and test vectors to detect possible manufacturing faults. It reports test coverage and stores all generated results. Basically, it makes the design test-ready by generating the data needed for DFT verification.

Pin Assignment File (.pinassign)
This file defines how the test pins are assigned for DFT. For example, it maps reset to test mode, SE to scan enable, clk as test clock, and scan_in / scan_out as scan data pins. These mappings help Modus and Xcelium know which pins are used for test control and scan operations during simulation and ATPG.

UART_Top.test_netlist.v (Gate-Level Netlist)
This is the final synthesized netlist of my UART design after DFT insertion. It includes all standard cells like NANDs, flip-flops, and multiplexers from the 65nm library. I can see how each flip-flop is connected with scan input, scan enable, and scan output. This file shows how my RTL design is transformed into an actual hardware-level representation that can be implemented on silicon and tested with scan chains.

Now, go to the output folder in your directory.

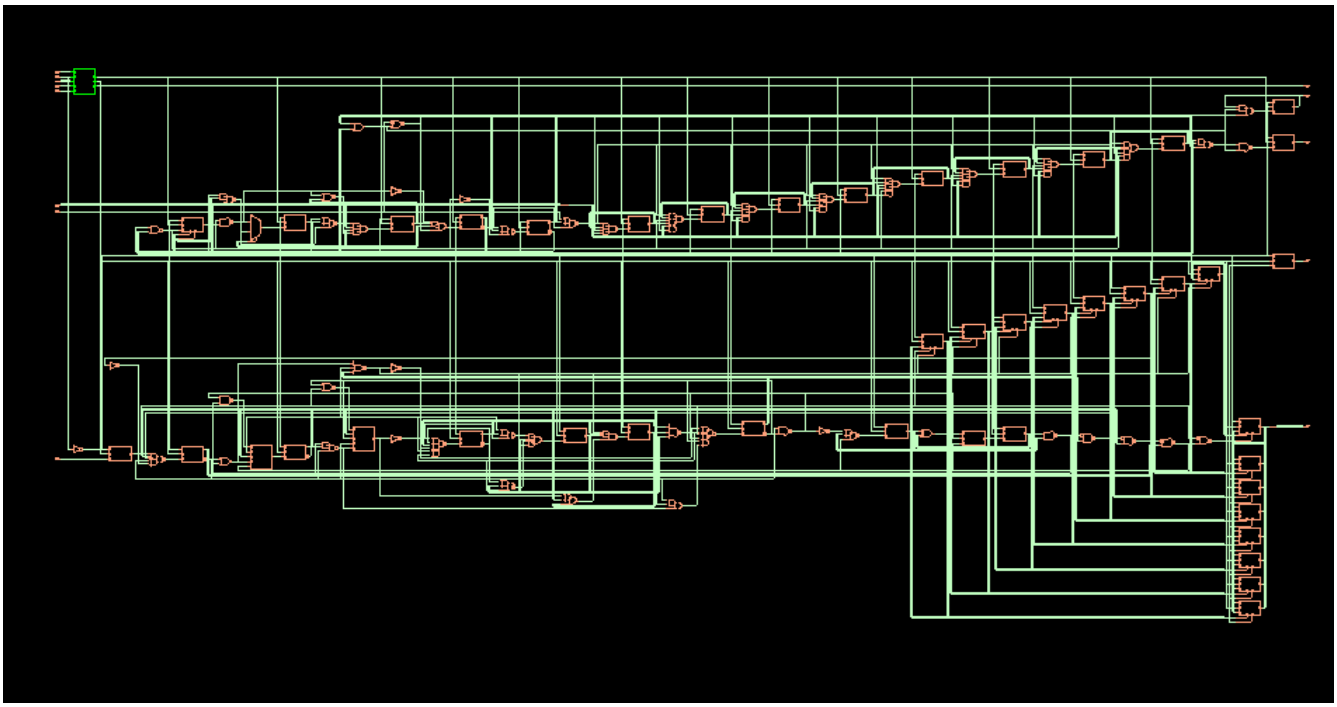


UART_Top.test_netlist.v (Gate-Level Netlist File)
This file represents the final synthesized netlist of my UART design after DFT insertion. It shows the complete hardware-level structure where all Verilog behavioral code is converted into standard cells like flip-flops, gates, and multiplexers from the 65nm library. It also includes scan-related pins such as scan_in, scan_out, and SE to make the design testable using scan chains. Each module like BaudRate and UART_Top is defined at the gate level, showing how data flows through logic cells during operation and testing. This file is mainly used for gate-level simulation, static timing analysis, and test verification after synthesis.

SCAN CHAIN (.scan) File
This file lists the scan chain configuration created during the DFT process. It defines how flip-flops are connected serially from scan_in to scan_out to form scan chains. Each flip-flop entry includes its scan input, output, and the clock domain it belongs to. The purpose of this file is to make sure all sequential elements can be accessed and tested during manufacturing. It also helps the ATPG tool generate and apply test patterns to verify that the chip works correctly after fabrication.

SDC (Timing Constraint) File
This file contains all the timing constraints for my UART design during synthesis and optimization. It includes commands like create_clock to define the main system clock period (10ns), and sets input/output delays, clock transition, and clock uncertainty. These constraints guide the synthesis tool to achieve timing closure by ensuring all paths meet setup and hold requirements. It also helps determine the maximum operating frequency of my design by adjusting the clock period or timing constraints.

Schematic view:

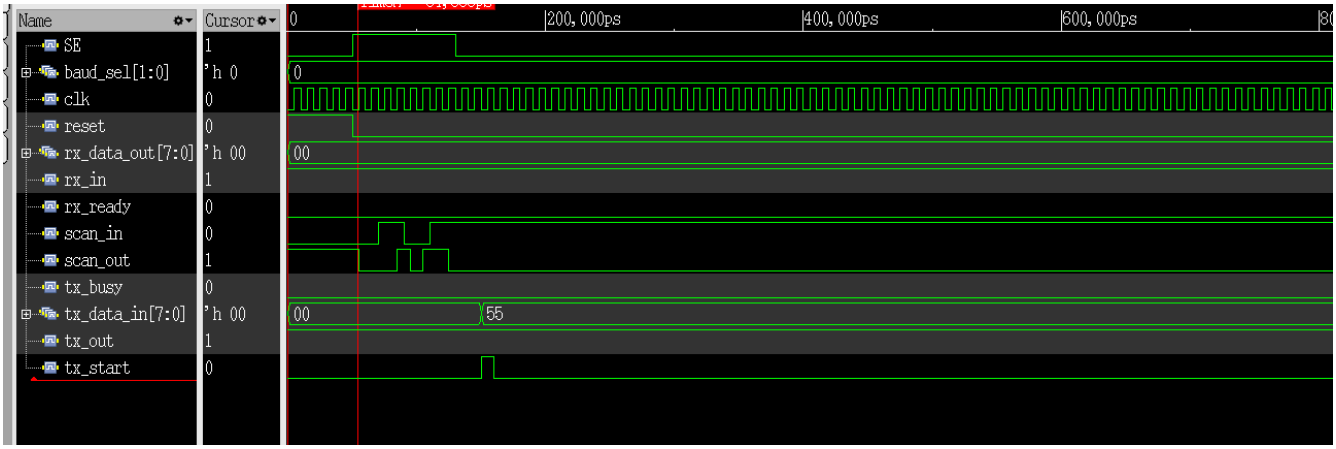


```
xmvlog_cg: *W,DLCPTH (/home/cc/mnt/XCELIUM2309/tools/inca/files/cdsvhdl.lib,15): cds.lib Inva
                worklib.Receiver:sv <0x441032fd>
                        streams:   9, words:  3810
                worklib.Transmitter:sv <0x059482c3>
                        streams:   9, words:  3143
                worklib.BaudRate:sv <0x32faee77>
                        streams:   7, words:  3102
        Building instance specific data structures.
        Loading native compiled code:     ..................... Done
        Design hierarchy summary:
                                  Instances  Unique
                Modules:                322      84
                UDPs:                  1498       3
                Primitives:            2163       8
                Timing outputs:         336      76
                Registers:              103      25
                Scalar wires:           539       -
                Vectored wires:           2       -
                Always blocks:            6       6
                Pseudo assignments:       1       -
                Timing checks:         3080       -
                Process Clocks:           5       3
                Simulation timescale:   1ps
        Writing initial simulation snapshot: worklib.BaudRate:v
DEFINE std       ./STD
```

Explanation:

This netlist now have the inputs of scan enables, scan_in as inputs and scan_out.

Waveform:



Explanation:

In my waveform, I first kept the UART in reset for some time to initialize everything. After that, I enabled the scan mode by setting SE = 1. During this time, I changed the scan_in values (0 and 1), and I could see these bits shifting through the scan flip-flops in the waveform. After a few clock cycles, the scan_out signal also changed, showing that the scan chain was working properly. Then I turned off scan mode (SE = 0) to go back to normal UART operation. In the functional part, I sent the data 0x55 for transmission. The waveform shows tx_busy going high when transmission started, and I could see serial bits appearing on tx_out. Since tx_out was connected to rx_in, the UART received the same data back. After the reception completed, rx_ready became high and rx_data_out showed 0x55, which confirms that both transmit and receive worked correctly and that the scan flip-flops did not affect the normal operation.

Critical Analysis:

In this lab, I performed the complete synthesis flow for the UART design, which included multiple RTL modules — baud rate generator, transmitter, receiver, and top module. Each module had to be verified and combined properly in the synthesis environment to ensure the design functioned as a single integrated UART system. The process required setting up libraries, constraints, and synthesis scripts carefully to achieve correct timing and area optimization.

The main focus of the lab was to understand how RTL code is transformed into a gate-level netlist using standard cells from the 65nm library. Through this process, I learned that even though the RTL design may function perfectly in simulation, the hardware implementation introduces real-world factors like cell delay, load, and wire capacitance, which can affect timing. The synthesis reports helped me analyze these parameters in terms of area, timing, power, and quality of results (QOR).

After synthesis, we did Gate-Level Simulation (GLS) using the SDF file to check real timing behavior. The same testbench was used with the $sdf_annotate command to add delay information. The simulation proved that the post-synthesis design worked the same as the RTL design in both normal mode and scan mode.

Overall, this lab helped us understand the full ASIC design flow — from RTL design, synthesis, and DFT scan insertion to gate-level simulation. It showed that adding scan flip-flops improves testability without affecting the design's timing, area, or functionality too much.